

React Manual Testing, Bug Fixing & UI Validation

Answers for Tasks 1–12

Note: Tasks 1–3, 4–5, 7–8, and 10–11 contain repeated or closely related requirements. They are answered individually where appropriate, while the repeated implementation uses the same tested solution.

Task 1: Flipkart-style Add to Cart — manual test and bug fix

Test procedure: Open the product page, click **Add to Cart** repeatedly (for example 3–5 times), then inspect the cart. The expected result is one product entry and a cart count of 1 for that product.

Bug noticed: The original implementation could push the same product into the cart every time the button was clicked, creating duplicate entries and increasing the cart count incorrectly.

Fix: Check whether the product already exists in the cart before adding it. If it exists, do not push it again.

```
const [cart, setCart] = useState([]);

function addToCart(product) {
  setCart(prevCart => {
    const alreadyAdded = prevCart.some(item => item.id === product.id);
    if (alreadyAdded) return prevCart;
    return [...prevCart, product];
  });
}
```

Task 2: Flipkart-style Add to Cart — repeated-click verification

Manual test: Click the same Add to Cart button 5 times quickly.

Expected: The product appears only once and the cart count remains 1.

Observed after the fix: One product entry remains in the cart. Repeated clicks do not create duplicates.

Additional check: Click a different product. The cart count should become 2, showing that the duplicate-prevention logic is based on product ID rather than blocking all additions.

```
function addToCart(product) {
  setCart(prev => {
    if (prev.some(item => item.id === product.id)) {
      return prev;
    }
    return [...prev, product];
  });
}
```

Task 3: Add to Cart — cart count updates correctly

Test cases: (1) Initial cart count = 0. (2) Click Product A once → count = 1. (3) Click Product A again → count stays 1. (4) Click Product B → count = 2. (5) Click Product B again → count stays 2.

Result: The count is derived from the cart array, so it updates automatically when the state changes.

```
const cartCount = cart.length;

return (
  <>
    <button onClick={() => addToCart(product)}>
      Add to Cart
    </button>
    <span>Cart ({cartCount})</span>
  </>
);
```

Task 4: Login form with email and password validation

The form uses React state for both fields. When Login is clicked, it checks whether email or password is empty and conditionally displays an error below the relevant input.

Expected behavior: Empty email → email error. Empty password → password error. Both filled → no empty-field errors.

```
import { useState } from "react";

export default function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [errors, setErrors] = useState({});

  function handleLogin(e) {
    e.preventDefault();

    const newErrors = {};
    if (!email.trim()) newErrors.email = "Email is required.";
    if (!password.trim()) newErrors.password = "Password is required.";

    setErrors(newErrors);

    if (Object.keys(newErrors).length === 0) {
      console.log("Login submitted");
    }
  }

  return (
    <form onSubmit={handleLogin}>
      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={e => setEmail(e.target.value)}
      />
      {errors.email && <p>{errors.email}</p>}

      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={e => setPassword(e.target.value)}
      />
      {errors.password && <p>{errors.password}</p>}

      <button type="submit">Login</button>
    </form>
  );
}
```

Task 5: Login form — validation test cases

Test 1: Submit with both fields empty → two required-field messages appear.

Test 2: Enter email only → password error appears.

Test 3: Enter password only → email error appears.

Test 4: Enter both values → validation errors disappear and the form can proceed.

Bug prevention: `preventDefault()` stops an invalid form from causing an unwanted page refresh.

Task 6: Spotify-style Create Playlist form

Test each field by leaving it blank and submitting. The form should reject an empty playlist name or description and display an error below the corresponding field.

Result: Basic state-based validation prevents incomplete playlist data from being submitted.

```
import { useState } from "react";

export default function PlaylistForm() {
  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [errors, setErrors] = useState({});
```

```

function handleSubmit(e) {
  e.preventDefault();
  const newErrors = {};

  if (!name.trim()) newErrors.name = "Playlist name is required.";
  if (!description.trim()) {
    newErrors.description = "Description is required.";
  }

  setErrors(newErrors);

  if (Object.keys(newErrors).length === 0) {
    const playlist = { name: name.trim(), description: description.trim() };
    console.log("New playlist:", playlist);
  }
}

return (
  <form onSubmit={handleSubmit}>
    <input
      value={name}
      onChange={e => setName(e.target.value)}
      placeholder="Playlist name"
    />
    {errors.name && <p>{errors.name}</p>}

    <textarea
      value={description}
      onChange={e => setDescription(e.target.value)}
      placeholder="Description"
    />
    {errors.description && <p>{errors.description}</p>}

    <button type="submit">Create Playlist</button>
  </form>
);
}

```

Task 7: Zomato-style Add Review — fix blank textarea submission

Bug: The form submits even when the review textarea contains only an empty string or spaces.

Fix: Trim the review text before validating. If the trimmed value is empty, set an error and return without submitting.

```

import { useState } from "react";

function ReviewForm() {
  const [review, setReview] = useState("");
  const [error, setError] = useState("");

  function handleSubmit(e) {
    e.preventDefault();

    if (!review.trim()) {
      setError("Review cannot be blank.");
      return;
    }

    setError("");
    console.log("Review submitted:", review.trim());
  }

  return (
    <form onSubmit={handleSubmit}>
      <textarea
        value={review}
        onChange={e => setReview(e.target.value)}
        placeholder="Write your review"
      />
      {error && <p>{error}</p>}
      <button type="submit">Submit Review</button>
    </form>
  );
}

```

Task 8: Zomato-style Add Review — verification after the fix

Test 1: Submit with an empty textarea → submission is blocked and “Review cannot be blank.” is shown.

Test 2: Enter only spaces → submission is also blocked because trim() removes surrounding whitespace.

Test 3: Enter a valid review → the error clears and the review is accepted.

Result: The blank-review bug is fixed.

Task 9: Fix addFollower(name) so invalid names are rejected

The function should ignore null, undefined, empty strings, and strings containing only spaces. Valid names are trimmed before being stored.

```
const followers = [];  
  
function addFollower(name) {  
  if (typeof name !== "string") return;  
  
  const cleanName = name.trim();  
  if (!cleanName) return;  
  
  followers.push(cleanName);  
}  
  
// Examples:  
addFollower(null);           // ignored  
addFollower("");            // ignored  
addFollower(" ");           // ignored  
addFollower(" Rahul ");     // adds "Rahul"  
  
console.log(followers);    // ["Rahul"]
```

Task 10: Main-screen React testing checklist — at least 5 common bugs

Use this checklist while manually testing buttons and form fields:

#	Common bug	How to test	How to fix
1	Double add	Click Add to Cart several times.	Check existing product ID before pushing to state.
2	Empty fields accepted	Submit each form with one field blank.	Validate with trim() and conditionally render errors.
3	Cart count incorrect	Add the same product repeatedly and add another product.	Derive count from the unique cart state.
4	Page refresh on submit	Submit a form and observe whether the page reloads.	Call e.preventDefault() in the submit handler.
5	Whitespace input accepted	Enter only spaces into text fields.	Use value.trim() before accepting input.
6	Error message does not clear	Fix an invalid field after an error is shown.	Recalculate/reset validation errors on each submit/change.
7	UI state not updated	Change an input and inspect displayed values.	Use controlled inputs with value and onChange.

Task 11: Main-screen testing — concise execution report

Testing completed: Buttons were tested with single clicks and repeated clicks; text inputs were tested with valid, empty, and whitespace values; forms were submitted with different combinations of data.

Five key bugs and fixes: duplicate cart additions → check product ID; empty form fields → required-field validation; incorrect cart count → derive count from state; unwanted refresh → preventDefault(); whitespace-only data → trim input.

Acceptance result: After applying these fixes, repeated Add to Cart clicks create one entry per product, invalid forms are blocked, and validation messages are displayed near the relevant fields.

Task 12: Zomato-style review form — full testing and validation

Test scenario A — valid data: Enter a valid review and a rating from 1 to 5, then submit. The form should accept the data.

Test scenario B — missing review: Leave the review blank and submit. The form should block submission and show a required-review message.

Test scenario C — invalid rating: Try a rating below 1 or above 5, such as 6. The form should block submission and show a rating-range message.

Bug found and fixed: The original form allowed blank reviews. The fix validates `review.trim()` before submission.

Recommended complete validation: Validate both the review and rating before accepting the form.

```
function validateReview(review, rating) {
  const errors = {};

  if (!review.trim()) {
    errors.review = "Review is required.";
  }

  const numericRating = Number(rating);
  if (!Number.isInteger(numericRating) ||
    numericRating < 1 || numericRating > 5) {
    errors.rating = "Rating must be a whole number from 1 to 5.";
  }

  return errors;
}

function handleSubmit(e) {
  e.preventDefault();

  const errors = validateReview(review, rating);
  setErrors(errors);

  if (Object.keys(errors).length === 0) {
    console.log("Review submitted successfully.");
  }
}
```

Final Summary

The tasks demonstrate practical React UI testing and basic bug fixing. The main techniques are controlled inputs, state-based validation, `trim()` for text normalization, checking unique product IDs before adding to a cart, deriving UI counts from state, and `preventDefault()` for safe form submission. These checks help prevent duplicate records, incomplete submissions, incorrect counts, and common UI validation issues.

Expected manual-test outcome: repeated clicks do not duplicate the same product; cart count reflects unique products; empty fields show useful errors; blank reviews are rejected; ratings outside 1–5 are rejected; and valid submissions are accepted.